

ML Challenge 2026: Business Entity Resolution

Comprehensive End-to-End Pipeline & Step-by-Step Technical Guide

1. Executive Overview & Problem Definition

The Problem: Modern commercial platforms ingest merchant identity records from multiple independent data sources (Source 1, Source 2, and Source 3). These records describe real-world businesses with severe typographical noise, legal suffix discrepancies ('Pvt Ltd' vs 'Limited'), partial/missing addresses, and missing postal codes, with **no common shared identifiers**.

The Goal: For each reference business entity in **Source 1** (over 1.73 Million entities), identify all matching counterpart records in **Source 2** and **Source 3** (over 9.97 Million candidate records) across the US, India, and France.

The Scale & Evaluation Challenge:

- **Scale:** A naive pairwise comparison would require $1.73 \times 10^6 \times 9.97 \times 10^6 \approx 1.72 \times 10^{13}$ (17.2 Trillion) string comparisons.
- **Metric ($F_{0.5}$):** The evaluation metric weights precision **2x higher than recall**. A single incorrect merge reduces that entity's score to 0.0.

2. Step-by-Step Pipeline Walkthrough

Step 1: Text Normalization & Legal Cleaning

Cleans and standardizes raw noisy text before comparison.

- Strips country-specific legal corporate suffixes (e.g. 'Pvt Ltd', 'Limited', 'Corporation', 'GmbH', 'SARL', 'LLC', 'PLC').
 - Normalizes ampersands ('&' -> 'and'), removes punctuation, converts all text to lowercase, and extracts informative character tokens (≥ 3 chars).
- **Why:** Ensures 'Acme Corp.' and 'Acme Corporation' are recognized as the exact same core business name.

Step 2: Candidate Generation (Inverted Index Blocking)

Reduces the massive 17-Trillion search space down to high-probability pairs in seconds.

- Constructs an Inverted Token Index over all 9.97M records in Source 2 & 3, mapping distinctive words to candidate IDs.
 - Frequency Capping: Tokens appearing in >100 records are capped to eliminate generic words (e.g., 'trading', 'services').
 - Country Partitioning: Restricts candidates strictly to the same country (US, India, France).
 - Top- K Selection: Ranks candidates by keyword overlap and extracts the Top-10 candidates per Source 1 entity.
- **Why:** Eliminates 99.999% of impossible pairs, achieving $>99.9\%$ blocking recall while keeping execution under minutes.

Step 3: Pairwise Feature Engineering (19 Signals)

Transforms text pairs into rich numerical similarity vectors for Machine Learning.

- **Name Similarities (8 features):** Levenshtein edit distance, Token Sort Ratio, Token Set Ratio, Partial Ratio, Token Jaccard similarity, length difference, first-word exact match, prefix containment.
- **Address Similarities (5 features):** Address Levenshtein distance, Token Sort Ratio, Token Jaccard overlap, Digit/Number Jaccard overlap (street numbers & PIN codes), length difference.
- **Group-Relative Features (5 features):** Composite quick score, candidate group size, intra-group rank, score gap to #1 candidate, is-top-1 indicator.
- **Country Match (1 feature):** Exact country match indicator.
- **Why:** Group-relative features allow the ML model to evaluate if a candidate is significantly better than its peers.

Step 4: Gradient Boosted Matching Model (LightGBM)

Trains an AI model to predict exact match probabilities for candidate pairs.

- Uses LightGBM (Gradient Boosted Decision Trees) configured with `num_leaves=31`, `learning_rate=0.04`, and `is_unbalance=True`.
 - **GroupKFold Validation:** Evaluates performance using 5-fold cross-validation grouped strictly by `source1_entity_id`.
 - **Why GroupKFold:** Prevents data leakage by ensuring the model is always tested on completely unseen companies.

Step 5: Macro F0.5 Threshold Optimization

Selects the mathematical probability cutoff that maximizes the competition score.

- Sweeps decision thresholds from 0.50 to 0.99 over Out-of-Fold (OOF) cross-validation predictions.
 - Identifies the optimal threshold at **0.94**, achieving a macro **$MF_{0.5} = 0.9654$** .
 - Singleton Handling: Entities with no candidate exceeding 0.94 are output as empty singletons (earning 1.0 accuracy).
 - **Why:** High threshold strictly prevents false merges, directly catering to the $MF_{0.5}$ metric penalty structure.

Step 6: High-Speed Streaming Test Inference

Runs end-to-end inference on the full 1.73M test set in a single continuous stream.

- Pure Python line-by-line file streaming eliminates pandas memory overhead.
 - Batches candidates in chunks of 10,000 for lightning-fast multi-threaded LightGBM C++ predictions.
 - Flushes scored results directly to output/matching_results.tsv and output/candidate_pairs.tsv.
 - **Why:** Resolved all 1.73M records in ~12 minutes using only 4GB RAM without any crashes or paging.

Step 7: Automated Validation & Submission Packaging

Validates output files and builds the final submission package.

- Executes the organizers' official validator (utils/validate_submission.py) → **PASS (exit 0)**.
 - Verifies 100% test entity coverage (exactly 1,732,544 rows, 0 duplicate IDs, 0 formatting errors).
 - Compresses code, outputs, and documentation into final_submission_package.zip.

Step 8: AWS S3 Artifact Sync (Cloud Integration)

Optionally downloads datasets from S3 before the run and uploads outputs + model after completion.

- **Download (pre-run):** aws_utils.download_dataset_from_s3() pulls all 7 TSV dataset files from s3://<BER_S3_BUCKET>/<prefix>/dataset/ into the local dataset/ directory, skipping files already present.
- **Upload (post-run):** aws_utils.upload_artifacts_to_s3() pushes model.txt, matching_results.tsv, and candidate_pairs.tsv to s3://<BER_S3_BUCKET>/<prefix>/artifacts/.
- **Zero-config fallback:** Both functions are no-ops when BER_S3_BUCKET env-var is unset — the pipeline runs identically without AWS credentials.
- **Why:** Enables the pipeline to run on any cloud VM (EC2, SageMaker) without manual data transfer, and persists trained models centrally in S3 for reproducibility.

3. Technology Stack & Role Summary

Technology	Exact Purpose	How it Operates
LightGBM	Pairwise Match Classifier	Tree-based ensemble scoring 19 non-linear similarity and group-rank features.
RapidFuzz	C++ String Distance Engine	Computes Levenshtein and token-set ratios at over 50,000 pairs/sec.
Inverted Index	High-Recall Blocker	Keyword-to-ID lookup with frequency capping and country partitioning.
Scikit-Learn	Cross-Validation & Metrics	5-Fold GroupKFold partitioning to prevent entity-level data leakage.
Streaming I/O	Big Data Execution Engine	Line-by-line buffered file reading and writing directly to disk.
AWS S3 (boto3)	Cloud Dataset & Artifact Store	Downloads raw TSVs from S3 before the run; uploads model.txt and output TSVs after the run. Optional — no-op when BER_S3_BUCKET is unset.

4. Validation Results & Deliverables

- **Validation $MF_{0.5}$ Score:** **0.9654** at optimal probability threshold **0.94**.
- **Official Validator Status:** **PASS (Exit Code 0)** — Zero errors or blocking issues.
- **Test Set Coverage:** Exactly **1,732,544 rows** in both output files (310,707 matches, 1,421,837 singletons).
- **Deliverables Generated:**
 1. output/matching_results.tsv (32.8 MB) — Scored leaderboard submission file.

2. output/candidate_pairs.tsv (247 MB) — Blocking stage candidate pairs.
3. final_submission_package.zip (61.1 MB) — Complete reproducible submission archive.
4. Documentation_template.md — Detailed technical methodology report.

5. AWS S3 Integration

The pipeline includes an optional AWS S3 integration layer implemented in src/aws_utils.py. It is activated exclusively via environment variables — when not set, the pipeline behaves identically to its original local-only mode.

Environment Variable	Default	Description
BER_S3_BUCKET	(unset — S3 disabled)	S3 bucket name. Setting this enables all S3 steps.
BER_S3_PREFIX	business_entity_resolution	Key prefix inside the bucket for all dataset and artifact paths.
AWS_REGION	us-east-1	AWS region for the boto3 S3 client session.

Expected S3 Layout (under prefix):

- business_entity_resolution/dataset/train/train_source1.tsv
- business_entity_resolution/dataset/train/train_source2.tsv
- business_entity_resolution/dataset/train/train_source3.tsv
- business_entity_resolution/dataset/train/train_ground_truth.tsv
- business_entity_resolution/dataset/test/test_source1.tsv
- business_entity_resolution/dataset/test/test_source2.tsv
- business_entity_resolution/dataset/test/test_source3.tsv

Artifacts Uploaded After Each Run:

- business_entity_resolution/artifacts/model.txt
- business_entity_resolution/artifacts/output/matching_results.tsv
- business_entity_resolution/artifacts/output/candidate_pairs.tsv